Leçon 5 : Upload d'image

Ce TP a pour objectif de réaliser l'upload d'une image d'un film pendant son ajout puis l'affichage de l'image dans une vue thymeleaf. L'image uploadé sera gérée selon 2 façons :

- L'image sera stockée localement dans le dossier `Resources` du projet Spring Boot, son nom sera stocké dans la table film.
- L'image sera stockée sur le serveur `Cloudinary` et son URL sera stocké dans la table film.

1ère façon : gestion locale de l'image

Etape 1 : Changement de l'entité Film et du formulaire d'ajout d'un film

On commence par ajouter un attribut `photo` dans l'entité `Film` de type String. Cet attribut correspondra à la colonne `photo` dans la table `film` qui contiendra le nom de l'image :

```
private String photo;
```

Au prochain redémarrage du projet la structure de la table `film` sera mis à jour.

Ensuite on va ajouter dans le formulaire d'ajout d'un produit `ajout.html` l'attribut `enctype` dans la balise `form` :

```
<form action="/produit/add" method="post" enctype="multipart/form-data">
```

Explication :

L'attribut `enctype="multipart/form-data"` dans la balise HTML `<form>` est utilisé pour spécifier comment les données du formulaire doivent être encodées et transférées vers le serveur lorsqu'un utilisateur soumet le formulaire.

Cela signifie que les données seront envoyées en plusieurs parties (multipart), et non pas sous forme d'une simple chaîne de caractères. Cet encodage est indispensable lorsqu'un formulaire contient un champ de type file.

Avec multipart/form-data :

- Chaque champ du formulaire est envoyé comme une partie séparée
- Les fichiers sont envoyés en binaire
- Le serveur peut récupérer le contenu du fichier + son nom + son type

Ajouter dans le même formulaire un input de type `file` pour uploader une fichier image :

```
<label class="form-label">Photo: </label>
<input type="file" name="file" accept="image/png, image/jpeg" class="form-control" />
```

Noter bien que le nom de l'input n'est pas `photo` comme le nom de l'attribut dans l'entité `Film`. Ceci permettra de le récupérer séparément dans le contrôleur.

Etape 2 : Changement dans le contrôleur

Dans le contrôleur `FilmController`, on va définir le chemin du dossier dans lequel seront uplodées les photos. Toutes les ressources web statiques comme les images, fichiers css et js doivent être placé par défaut dans le dossier `static` sous le dossier `resources`.

```
private String uploadDirectory = System.getProperty("user.dir")+"\\src\\main\\resources\\static\\photos";
```

`System.getProperty("user.dir")` : retourne le chemin sur disque de la racine du projet courant (chemin du répertoire du projet). Le chemin des images est complété jusqu'à le dossier `photos` **(à créer)** sous `static`.

Toujours dans le même contrôleur, on va modifier les paramètres de l'action `add` comme suit :

```
@PostMapping("add")
public String add(Film f, @RequestParam("file") MultipartFile multipartFile) {
```

On a ajouté un paramètre de type `MultipartFile` qui recevra le fichier uploadé (la photo) à partir de l'input `file` du formulaire d'ajout.

Le nouveau code de l'action `add` devrait suivre les étapes suivantes (toutes les nouvelles importations doivent se faire à partir de `java.nio.file.*`) :

- On récupère dans une variable `fileName` le nom original de la photo en appelant la méthode `getOriginalFileName()` de l'objet `multipartFile`.
- On définit dans une variable de type `Path` le chemin complet où sera sauvegardée l'image. Ce chemin est obtenu par l'appel de la méthode statique `get` de la classe `Paths` avec les paramètres `uploadDirectory` et le nom original de la photo `fileName`.
- Ensuite on écrit physiquement le fichier de l'image avec son chemin complet à partir de la source binaire de ce fichier : `multipartFile.getBytes()`. L'écriture se fait avec la méthode statique `write` de la classe `Files`. Cette écriture doit se faire dans un bloc `try-catch`.
- sans oublier d'affecter dans l'attribut `photo` du film le nom original de l'image avant de créer le film `f`.

Test l'upload d'une image d'un film. Vérifier l'existence de l'image uploadée dans le dossier `photos`.

Etape 3 : Affichage de l'image dans une vue Thymeleaf

Dans la vue `affiche.html` On va ajouter dans le tableau une colonne pour afficher une miniature de l'image pour chaque film.

Pour les films qui n'ont pas de photo, on peut afficher, comme remplacement, une autre image standard placé dans le dossier photos nommée `no-image.jpg`, en faisant le test comme suit :

```
<span th:if="condition pas de photo" > afficher une image no-image.jpg </span>
<span th:unless="condition pas de photo" > afficher la photo du film </span>
```

Ajouter dans le fichier application.properties les 2 propriétés suivantes :

```
spring.web.resources.static-locations[0]=file:src/main/resources/static/
spring.web.resources.static-locations[1]=classpath:/static/
```

Etape 4 : Effectuer les modifications nécessaires dans la vue et l'action de modification d'un film en tenant compte (ou pas) de l'upload d'image au cours de la modification.

2ème façon : gestion de l'image avec Cloudinary

Dans cette 2ème façon on va conserver l'étape 1 précédente puis on va suivre les étapes suivantes :

➢ Créer un compte sur le serveur Cloudinary et récupérer les informations suivantes : cloud name, API Key et API Secret.

➢ Rechercher dans MavenRepository la description de la dépendance `cloudinary http44` puis l'ajouter dans le fichier `pom.xml` et valider son changement.

➢ Ajouter dans le fichier `application.properties` les propriétés suivantes :

```
cloudinary.cloud_name=VALEUR_CLOUD_NAME
cloudinary.api_key=VALEUR_API_KEY
cloudinary.api_secret=VALEUR_API_SECRET
```

➢ Créer dans un package `config` la classe de configuration `CloudinaryConfig` qui déclare 3 attributs recevant leurs valeurs des propriétés déclarées puis génère avec une méthode un Bean (Objet placé dans le context de Spring Boot pour faciliter son injection des dépendances). Ce Bean sera utilisé dans le contrôleur pour appliquer l'upload de l'image :

```
@Configuration
public class CloudinaryConfig {

    @Value("${cloudinary.cloud_name}")
    private String cloudName;

    @Value("${cloudinary.api_key}")
    private String apiKey;

    @Value("${cloudinary.api_secret}")
    private String apiSecret;

    @Bean
    public Cloudinary cloudinary() {
        return new Cloudinary(ObjectUtils.asMap(
                "cloud_name", cloudName,
                "api_key", apiKey,
                "api_secret", apiSecret
        ));
    }
}
```

➢ Dans `FilmController` injecter une dépendance de la classe `Cloudinary` et modifier l'action d'ajout d'un nouveau Film en remplaçant (<u>pour l'instant</u>) le code d'upload précédent par :

```
Map uploadResult = cloudinary.uploader().upload(multipartFile.getBytes(),
ObjectUtils.emptyMap());
String photo = uploadResult.get("url").toString();
film.setPhoto(photo);
iServiceFilm.createFilm(film);
...
```

➢ Modifier le code dans la vue `affiche` qui selon la façon d'upload de l'image celle-ci sera affichée (tester sur la valeur de début du nom de la photo).

➢ Ajouter dans le formulaire d'ajout d'un film 2 boutons radio pour choisir la destination de l'image (local ou cloud). Dans l'action de l'ajout d'un nouveau film utiliser la valeur de ce champ pour activer l'un des 2 traitements : enregistrement local de l'image ou bien sur Cloudinary.